

Advanced Cryptography

Elisabeth Oswald

MPC via Shamir Secret Sharing

ROADMAP

Conceptual View

Multiparty computation

MPC Basic Operations

Addition

Multiplication by a constant

General multiplication

MPC for Arithmetic Circuits

BGW

SETTING FOR MULTIPARTY COMPUTATION

Multiple parties P_i , with secret inputs X_i , wish to compute some shared function $F(X_1, X_2, \dots)$.

If the function requires the input of all parties, then all parties must participate in the computation. However it may be desirable that the result can be revealed by just a subset of parties.

Therefore some MPC schemes are realised via initially secret sharing the inputs.

But how do we compute on shares?

MPC VIA SECRET SHARING

Firstly: we have to consider how to map, what we want to compute (e.g. an average, a linear regression over samples, the product over inputs) into an **arithmetic circuit**.

What?? Assume that each party holds real valued inputs, and we want to compute some basic statistics over these inputs: this is not compatible with arithmetic in $\mathbb{Z}_p$. We first need to “convert” numbers in $\mathbb{R}$ to numbers in $\mathbb{Z}_p$! (Choose p to be very large.)

Then we need to represent the function as a sequence of operations over $\mathbb{Z}_p$: e.g. the average is a sum (i.e. a sequence of additions) that is scaled by a constant (the number of inputs).

PUTTING THIS TOGETHER

We express the function as an “ arithmetic circuit” with wires that represent values over $\mathbb{Z}_p$.

The secret input wires are shared out among all parties using Shamir’s scheme.

Then the parties compute the circuit, possibly by interacting (depending on the circuit) to jointly compute intermediate values and the final output.

In principle, the parties could do the computation “locally” (i.e. each party computes on their shares), but in practice, there is often a third party that provides the computation “as a service”.

EXAMPLE: ADDITION WITH THREE PARTIES

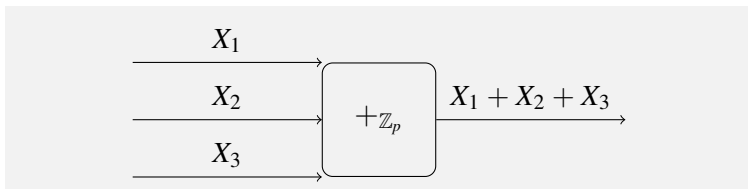


Figure: Modular addition with inputs from three parties: situation without secret sharing.

Suppose three parties wish to aggregate their inputs using a third untrusted party.

So in this example, the parties are not directly computing the sum, but another party is doing this for them.

EXAMPLE: MPC ADDITION WITH A $(2, 2)$ SECRET SHARING SCHEME

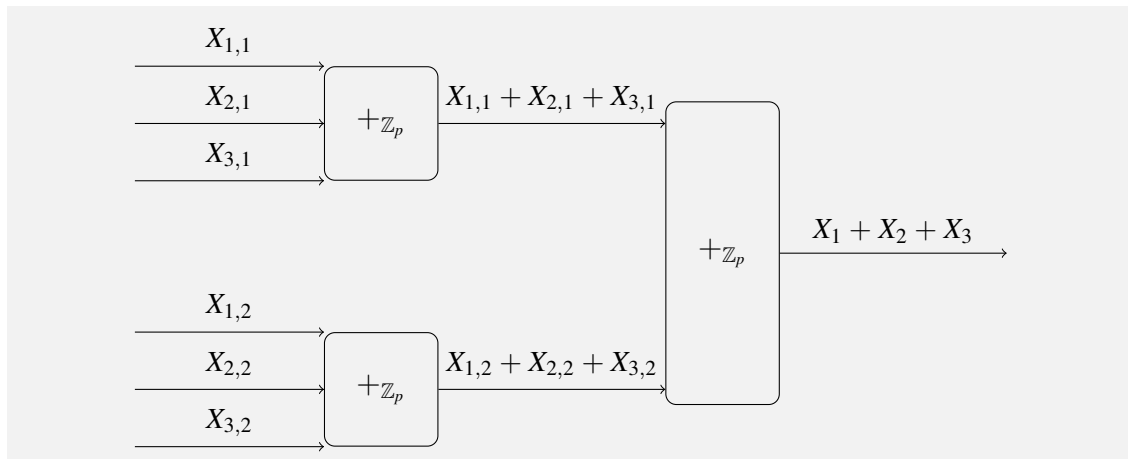


Figure: Modular addition with inputs from three parties using two independent aggregators.

EXAMPLE: MPC ADDITION WITH A $(3, 3)$ SECRET SHARING SCHEME

How would the computation look like if there were no aggregation servers, and the parties would perform everything locally?

ROADMAP

Conceptual View

Multiparty computation

MPC Basic Operations

Addition

Multiplication by a constant

General multiplication

MPC for Arithmetic Circuits

BGW

ADDITION WITH SHAMIR SHARES

Compute $c = a + b$ ($a, b, c \in \mathbb{Z}_p$).

Recall that:

$$f(x) = a + f_1x + f_2x^2 + \dots f_tx^t$$

$$g(x) = b + g_1x + g_2x^2 + \dots g_tx^t$$

The polynomial corresponding to $c = a + b$ is $h(x) = f(x) + g(x)$.

$h(x) = a + b + h_1x + h_2x + \dots h_tx^t$, whereby $h(0) = a + b$, and $h_i = f_i + g_i$. (We add the corresponding coefficients).

Because $f(i) + g(i) = (f + g)(i)$, each party “gets” $h(i)$ by simply adding the shares of the inputs that they have.

Notice that the addition modulo p is a permutation of $\mathbb{Z}_p$: it preserves the distribution of shares.

ADDITION: EXAMPLE WITH THREE PARTIES

We have $(3, 3)$ scheme and work over $\mathbb{Z}_{11}$.

Two values a and b are shared out.

The table below shows the situation from the perspective of the players (each player can only see the values in their column).

Value	P_1	P_2	P_3
a	9	2	3
b	6	0	7
$a + b$	4	2	10

The secret sum can be reconstructed by finding the unique polynomial that goes through $(1, 4)$, $(2, 2)$, and $(3, 10)$.

Or it can be constructed via the public recombination vector.

ROADMAP

Conceptual View

Multiparty computation

MPC Basic Operations

Addition

Multiplication by a constant

General multiplication

MPC for Arithmetic Circuits

BGW

MULTIPLICATION BY A CONSTANT

Notice that

$$c \cdot f(x) = c \cdot a_0 + c \cdot a_1 x^1 + \dots$$

Note that all the a_i were randomly chosen and the shares are also random numbers.

Therefore, $c \cdot f(i)$ is still random (multiplication over $\mathbb{Z}_p$ with a constant is a one-to-one mapping from $\mathbb{Z}_p^*$ to $\mathbb{Z}_p^*$).

ROADMAP

Conceptual View

Multiparty computation

MPC Basic Operations

Addition

Multiplication by a constant

General multiplication

MPC for Arithmetic Circuits

BGW

MULTIPLICATION

We want to do $a \cdot b \pmod{p}$.

$$f(x) = a + f_1x + f_2x^2 + \dots f_tx^t$$

$$g(x) = b + g_1x + g_2x^2 + \dots g_tx^t$$

The representation $c = a \cdot b$ is given by the product of the polynomials $f(x) \cdot g(x)$.

Thus $h(x) = a \cdot b + h_1x + h_2x^2 + \dots h_tx^{2t}$, whereby $h(0) = a \cdot b$.

Notice that because $f(i) \cdot g(i) = (f \cdot g)(i)$, each party “gets” $h(i)$ by simply multiplying their shares.

So far so good, but ...

MULTIPLICATION, CONT.

Notice the degree of $h(x)$:

$$h(x) = a \cdot b + h_1x + h_2x^2 + \dots h_tx^{2t}.$$

We need $2t$ players rather than t to recover the secret. Therefore if $t \geq n/2$, then reconstruction is in fact no longer possible.

Therefore we potentially must enlist more parties/use more shares if we want to do multiplications in our circuit.

But what if we want to do many multiplications? And are the shares still satisfying our security notion?

MULTIPLICATION, DEGREE REDUCTION

Assume now that we have enough parties/shares to reconstruct.

Each party takes their share $h(i)$ and produces a new (t, n) sharing for it.

This means, that P_i produces a degree t polynomial with $h(i)$ as the constant term, and randomly sampled coefficients otherwise.

They then give shares to all other parties.

This is where some (costly) interaction comes in. Once all parties have received all shares, each party reconstructs a new sharing for their share of the secret product.

DEGREE REDUCTION, CONT.

Slightly more mathematical: we have $h(0) = (f \cdot g)(0) = a \cdot b = c$; party P_i has $h(i)$.

The Lagrange reconstruction for $h(x)$ is $h(x) = \sum_{i=1}^n \delta_i(x)h(i)$. Thus $h(0) = \sum_{i=1}^n \delta_i(0)h(i)$, rewriting using $r_i = \delta_i(0)$ we have that $c = h(0) = \sum_{i=1}^n r_i \cdot h(i)$.

Degree reduction:

- ▶ Party P_i samples $\delta'_i(x)$ with degree $\leq t$ such that $\delta'_i(0) = h(i)$, and shares $\delta'_i(j)$ with party P_j .
- ▶ Party P_j computes $H(j) = \sum_{i=1}^n r_i \delta'_i(j)$.

Observe that $H(0) = \sum_{i=1}^n r_i \delta'_i(0) = \sum_{i=1}^n r_i h(i) = c$.

Therefore the points $H(i)$ are points on a polynomial H of degree at most t that evaluates to c on 0.

EXAMPLE

Three parties perform a multiplication of two values over $\mathbb{Z}_{11}$. They use a $(2, 3)$ scheme.

Value	P_1	P_2	P_3
a	2	10	7
b	0	9	7
$a \cdot b$	0	2	5

Can you reconstruct and work out the product?

Next, we do a degree reduction.

EXAMPLE, CONT.

The degree reduction works by creating new (t, n) sharings for the shares of the product that the parties P_i hold. In this example the players randomly select the polynomials:

$$P_1 : 0 + 3x$$

$$P_2 : 2 + 6x$$

$$P_3 : 5 + 2x.$$

With these they produce new shares, and distribute them:

Player				
P_1	3	8	7	$c_1 = 3$
P_2	6	3	9	$c_2 = 7$
P_3	9	9	0	$c_3 = 0$

Reconstruction works via: $c_1 \cdot 3 + c_2 \cdot 8 + c_3 \cdot 1 \pmod{11}$.

ROADMAP

Conceptual View

Multiparty computation

MPC Basic Operations

Addition

Multiplication by a constant

General multiplication

MPC for Arithmetic Circuits

BGW

BGW: Ben-Or, Goldwasser, and Wigderson

The protocol provides perfect security against passive adversaries: recall this implies that all users behave honestly, i.e. they follow the protocol (even when taken over by an adversary).

We assume that the function is given as a sequence of addition, multiplication by constant, and multiplication gates.

The sequence has a clear ordering, all parties know this ordering (i.e. they all compute the same function, in the same way).

For every multiplication gate, all parties must interact.

Input Sharing. Each party P_i with input x_i produces an (t, n) sharing of their input. They then distribute shares to the other parties.

Computation. Parties now process the circuit gate by gate, whereby they maintain the fact that if the inputs to a gate are (t, n) secret shared, then the outputs to the gate are also (t, n) secret shared. This requires no overhead for addition and multiplication by constant gates, but whenever a multiplication is required, then communication between all parties is required.

Output reconstruction. When all gates have been processed, the parties use Lagrange interpolation (i.e. the public reconstruction vector) to reconstruct the output. Therefore each party sends their share to all other parties (securely).

Input Sharing: perfectly secret because it is based on Shamir Secret Sharing.

Computation: each “gate” gates perfectly secret inputs and produces perfectly secret outputs (we argued this before).

Reconstruction: with the aid of the function output, call it y , can t corrupt parties learn anything “in addition” about the shares of the other parties? The intuitive answer is no because every step preserved the “random nature” of the shares.

In a real-random style proof we would need to show though how to produce the output sharing from the view of $t - 1$ corrupt parties. Notice that $t - 1$ corrupt parties can reconstruct the polynomial that represents the output y (because they have $t - 1$ shares and y), and therefore they can compute the output shares of the other (not corrupt parties). The simulator could do the same and therefore produce a “view” in the random world that is the same as what the adversary has in the real world.